

OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID

NQF 6

Software design and testing C#

SDT621

SUMMATIVE ASSESSMENT

2026



Table of Contents

DECLARATION OF AUTHENTICITY	3
QUALIFICATION AND ASSESSMENT DETAILS	4
Instructions:	5
Submission Requirements	5
Evidence Collection Requirements	5
To support your assessment submission, you must:	5
GitHub Submission Requirement	5
Summative Assessment Criteria:	6
Example of Collecting Evidence for your submission pdf:	6
SECTION A — Knowledge Questions (40 Marks)	7
True / False Questions [10]	9
SECTION B — Short Practical Questions (10 Marks)	10
Question 1	10
SECTION C: Scenario-Based Practical [50 Marks]	11
Question 1	11
SECTION C.2 — Button Event Logic Implementation (20 Marks)	11
SECTION C.3 — Software Testing Report (15 Marks)	12
SECTION C.4 — GitHub Version Control Evidence (5 Marks)	12
SECTION A — Knowledge Questions (40 Marks)	14
SECTION B — Short Practical Questions (10 Marks)	15
SECTION C — Scenario-Based Practical (50 Marks)	16
SECTION C.2 — Button Event Logic Implementation (20 Marks)	16
SECTION C.3 — Software Testing Report (15 Marks)	16
SECTION C.4 — GitHub Version Control Evidence (5 Marks)	17
SECTION C Mapping Summary	17

DECLARATION OF AUTHENTICITY

I (FULL NAME) | Aidan Hawley |

hereby declare that the contents of this assessment | **SDT621 SA** |
are entirely my own work, completed by me without any paraphrasing/copying, or presented as my own
work not accessed from any **AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.**

I have read and understood the assessment requirements and the Examination Rules and Regulations.

Signature: [Click or tap here to enter text.](#)

Date: 5/27/2026

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	SUMMATIVE ASSESSMENT
Hand Out Date	27 / 0 5 / 2026
Hand in Date	27 / 05 / 2026
Total Marks	100
Assessment Lead Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi

Student Name and Surname	Aidan Hawley
Student Number	20241039

Instructions:

Test or Debug Source Code to Ensure Client Needs Are Met

Please read the following instructions carefully before attempting this assessment:

- Read each question carefully and consider the mark allocation before answering.
- Answer all questions in the assessment.
- Provide clear, well-structured C# code and explanations where required.
- Apply your understanding of software testing and debugging techniques when solving the given scenarios.
- Ensure that your solutions meet the client requirements described in the scenario.
- Use meaningful variable names, proper formatting, and correct programming logic.
- Test your application before submission to confirm that it runs correctly.

Submission Requirements

- For your final submission, you must include:
- A signed Declaration of Authenticity (if not already included in the assessment document provided)
- Your completed Answer Sheet (PDF document) (preferably the official template provided with this assessment)
- Your Visual Studio project files (compressed as a ZIP file)

Evidence Collection Requirements

To support your assessment submission, you must:

- Take a screenshot of the program output for each question
- Ensure each screenshot shows the full screen, including the system date and time
- Ensure screenshots clearly demonstrate correct program execution
- Do NOT submit screenshots of your source code
- Submit your complete Visual Studio project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- Push your project code to GitHub
- Ensure the repository contains the correct and complete solution
- Share the GitHub repository link in your submission for verification and review
- Example: <https://github.com/your-username/project-name>

Summative Assessment Criteria:

Applied Knowledge

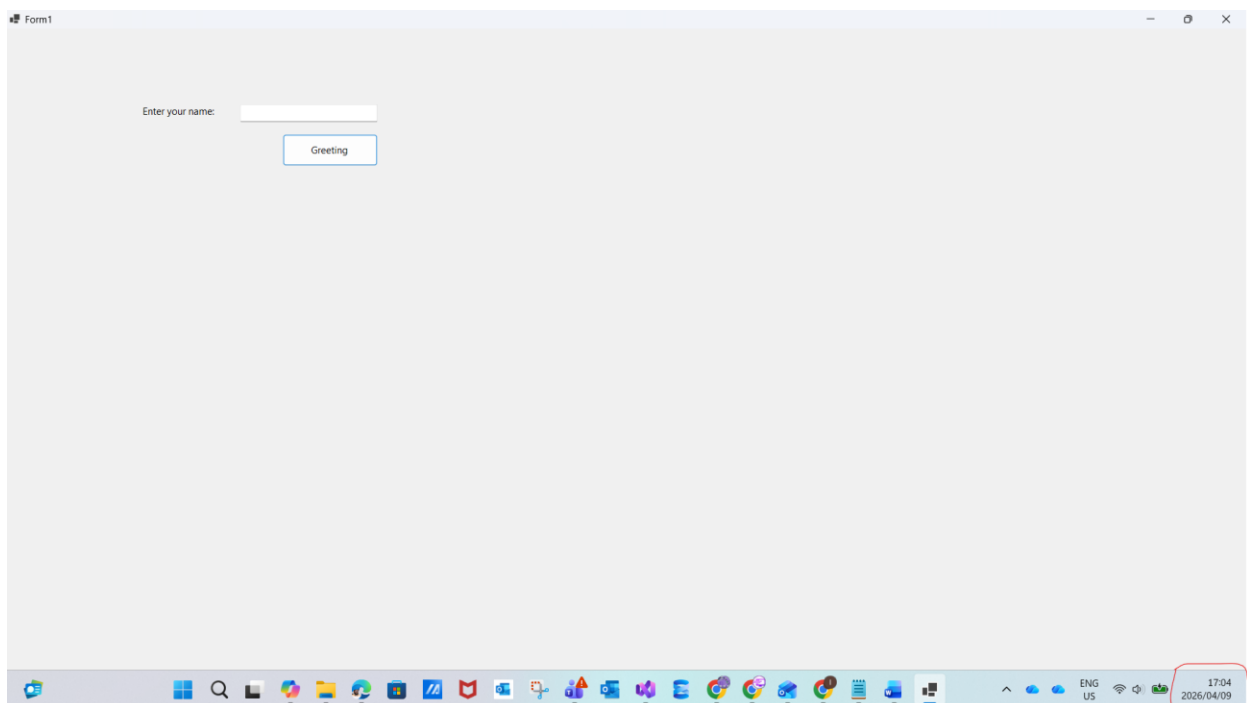
- AK0101 Knowledge of application testing techniques
- AK0102 Knowledge of test report writing

Internal Assessment Criteria

- IAC0101 functional and non-functional tests are identified
- IAC0102 The correct tests are applied
- IAC0103 A test report is produced

Example of Collecting Evidence for your submission pdf:

Question1 :



Ensure each screenshot shows the **full screen**, including the **system date and time**

SECTION A: Knowledge Questions (40 Marks)

Question 1

(30 Marks)

- 1.1 Define non-functional testing and explain what type of system qualities it evaluates. (2)
Functional testing used to test the systems performance, scalability and maintainability and ensures that the system runs optimally under different conditions.
- 1.2 Explain load testing as a non-functional testing type. (1)
Load testing is used to see how well the system runs under normal loads and conditions.
- 1.3 Describe the difference between load testing and stress testing in non-functional test design. (2)
Load testing tests the systems performance under normal conditions and work loads, where as stress testing tests the systems performance under extreme conditions and work loads.
- 1.4 Explain the difference between black-box testing and white-box testing. (3)
Black box testing is testing that is done with no knowledge of the existing code and its functions.
White box testing is testing that is done with knowledge and understanding of the systems existing code and functions.
- 1.5 Describe equivalence partitioning and explain how it improves functional testing efficiency (3)
Equivalence partitioning is when similar values are grouped together to allow for one value to be tested instead of all possible similar values. Equivalence partitioning improves functional testing efficiency by make it quicker to test a single value instead of all possible similar values, such as symbols.
- 1.6 A Loan Eligibility Processing System accepts customer income values between:
R1 000 and R150 000 Design TWO boundary value test cases to verify correct system behaviour. (4)
R1001 to check that the system is accepting values between R1000 and R150000. R150001 to check that the system is not accepting values that are greater than R150000.
- 1.7 Define error-case testing. (1)
Error case testing is when you deliberately enter the incorrect data type to see how the system handles this occurrence.
- 1.8 Explain why testing unexpected user inputs is important when validating software reliability. (2)
Testing unexpected user inputs is important as it shows how the system would handle its self if a user where to enter and incorrect data type, and prevents unepected or unhandled errors.
- 1.9 Provide TWO examples of error-case tests suitable for a login system. (2)
Testing that the usrename cannot contain spaces and that the username cannot be longer than 100 characters.
- 1.10 Define a test case in software testing. (2)
A test case is a plan of what is going to be tested, how it is going to be tested, what the expected resault is, what the actual resault is and whether the test passed or failed.
- 1.11List THREE components typically included in a structured test case. (3)
Decription, expected resault and test pass or fail.

1.12A testing session executed:

Test Cases	Passed	Failed
100	80	20

Prepare a structured test summary section for a system testing report including:

Test results overview

Recommendation to stakeholders (5)

A total of 100 test were performed of those 80 passed and 20 failed. I recommend that the failed test are fix before the system goes live or is made public, however enough of the test succeded and should not be a major concern if the system were to release or go live.

Total _____ / 30

True / False Questions

[10]

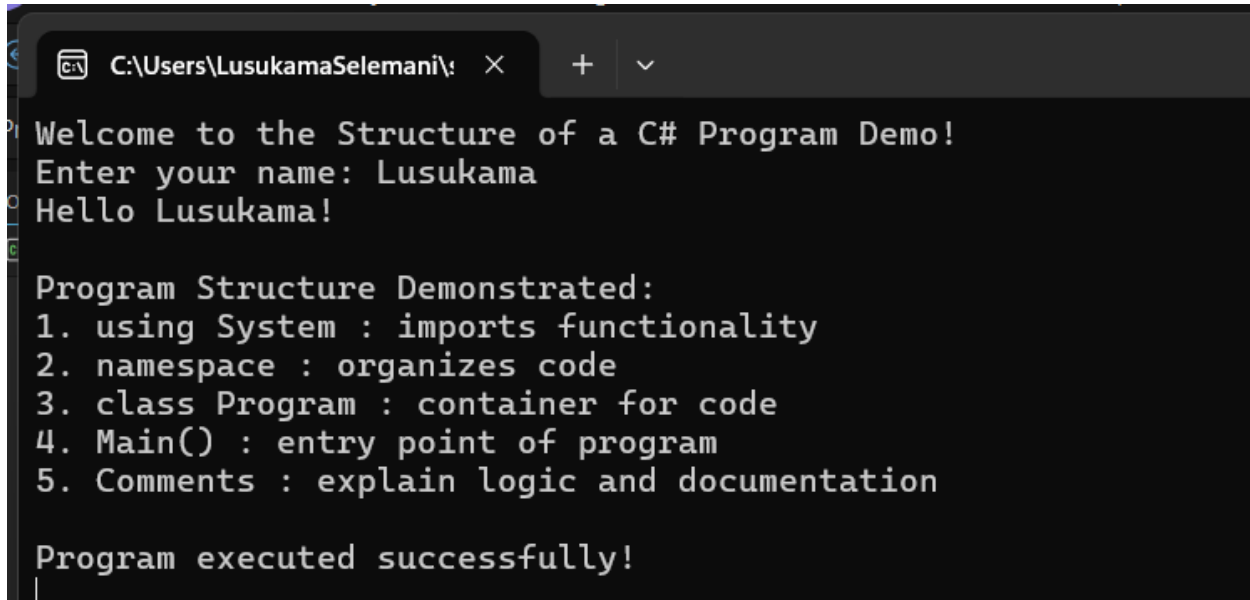
True / False Question	Answer
Exploratory testing allows testers to learn, design, and execute tests at the same time.	True
A good feature specification should be vague to allow flexibility during testing.	False
Specification review meetings help identify missing or incorrect requirements before testing begins.	True
Unit testing is typically performed after system testing.	False
Black-box testing focuses on testing system behavior without examining internal code structure.	True
White-box testing examines internal program logic and code structure.	True
Error-case testing helps ensure the system handles unexpected inputs correctly.	True
UX testing only evaluates system performance and ignores usability.	False
Security testing includes checking authentication and authorization mechanisms.	False
Maintainability testing helps determine how easily a system can be monitored and updated.	True

Total: ____ / 10

SECTION B: Short Practical Questions (10 Marks)

Question 1

1.1 Create a C# Console Application that the display the below output: (5)



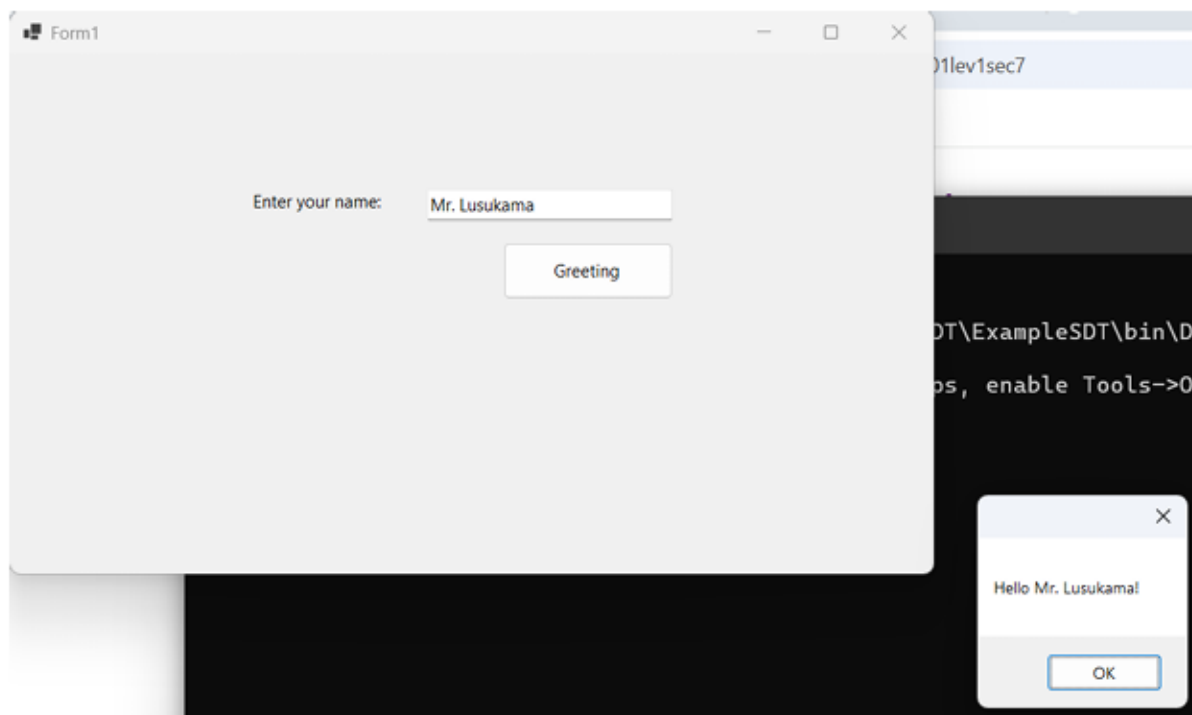
```

C:\Users\LusukamaSelemani\>
Welcome to the Structure of a C# Program Demo!
Enter your name: Lusukama
Hello Lusukama!

Program Structure Demonstrated:
1. using System : imports functionality
2. namespace : organizes code
3. class Program : container for code
4. Main() : entry point of program
5. Comments : explain logic and documentation

Program executed successfully!
  
```

1.2 Develop a Windows Forms Application that allows the user to enter their name and display a greeting message. (5)



SECTION C: Scenario-Based Practical [50 Marks]

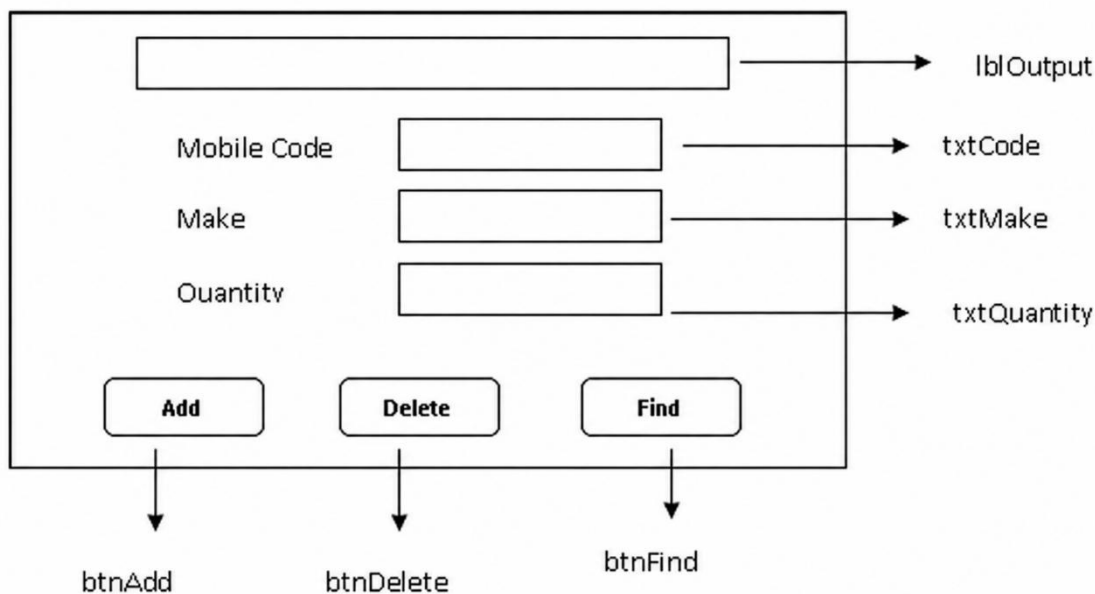
Question 1

You are working as a Junior Software Tester in a retail inventory company. The development team created a Mobile Stock Capture Windows Forms application, but the system contains interface naming issues, missing validations, and incorrect button behavior.

You are required to design, test, debug, and improve the Windows Forms application shown in the diagram.

SECTION C.1 — User Interface Design (10 Marks)

- 1.1 Create and design the following user interface using appropriate C# Windows Forms App controls and components. (10)



SECTION C.2 — Button Event Logic Implementation (20 Marks)

- 1.2 Write the events for each button

1.2.1 When the `btnAdd` button is clicked, the program should insert a record into the `tblMobilePhones` with the `MobileCode`, `Make` and `Quantity` from the corresponding data from the form and display the message "Record Added" in the `lblOutput`. (8)

1.2.2 When the `btnDelete` button is clicked, the program should delete the record whose `MobileCode` is typed in the `txtCode` text field and display the message "Record Found" in the `lblOutput`; or "Record NOT Found" if the record is not found. (7)

1.2.3 When the `btnFind` is clicked, the program should find the record whose `MobileCode` is typed in the `txtCode` text field and display the message "Record Deleted" in the `lblOutput`. (5)

Note: use functional and non-functional tests for this question

SECTION C.3 : Software Testing Report (15 Marks)

1.3 Produce a Software Testing Report

Using the results from Question 1, compile a professional software testing report for the Mobile Stock Capture Application.....

Your report must include the following sections:

1.3. 1 Application Description (3 Marks)

Provide a brief description of:

- Purpose of the application
- Main system functions
- User inputs captured

1.3.2 Test Cases Table (5 Marks)

Create a table similar to the example below:

Test Case	Input	Expected Result	Actual Result	Status

Include at least five test cases.

1.3.3 Identified Defects (4 Marks)

List any problems found during testing:

Delete function does not say that the record was deleted instead it says that the record was found. The find function says that the record was deleted instead of found. The add function adds a record even if it is already recorded.

1.3.4. Recommendations for Improvement (3 Marks)

Correct the message for the delete and find function. Make the add function only add if the record does not exist. Add a way for the system to display the record that was found.

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

1.3.5 Use of GitHub

To obtain the full 5 marks, learners must demonstrate correct use of GitHub version control by completing the following tasks:

- Create and work within a separate branch (not directly on the main branch)
- Make meaningful and professional commit messages that clearly describe the changes made
- Successfully merge the branch into the main (origin/main) branch after completing development
- Ensure the repository contains the complete working solution
- Submit a valid and accessible GitHub repository link

End Exam

SECTION A — Knowledge Questions (40 Marks)

Purpose of Section A

Assess theoretical understanding of testing techniques, validation strategies, and structured test documentation required before practical debugging begins.

QCTO Mapping

Question Area	Marks	QCTO Criterion	Evidence Demonstrated
Non-functional testing definition	2	AK0101	Knowledge of testing concepts
Load testing explanation	1	AK0101	Understanding performance testing
Load vs Stress testing comparison	2	AK0101	Distinguishing test strategies
Black-box vs White-box testing	3	AK0101	Test method classification
Equivalence partitioning	3	AK0101	Functional test design
Boundary value test cases	4	AK0101	Boundary testing application
Error-case testing definition	1	AK0101	Exception scenario awareness
Unexpected input validation importance	2	AK0101	Reliability assurance concepts
Login error-case examples	2	AK0101	Functional validation thinking
Definition of test case	2	AK0102	Test documentation knowledge
Components of test case	3	AK0102	Structured testing documentation
Test summary & stakeholder recommendation	5	AK0102	Test reporting interpretation

Section A Mapping Summary

Criterion	Marks
AK0101 Knowledge of testing techniques	25
AK0102 Knowledge of test report writing	15

Total Section A = 40 Marks

SECTION B — Short Practical Questions (10 Marks)

Purpose of Section B

Assess ability to implement and validate basic working software functionality before applying structured testing.

QCTO Mapping Table

Task	Marks	QCTO Criterion	Evidence Demonstrated
Console output application	5	IAC0102	Correct functional output validation
Windows Forms greeting application	5	IAC0102	UI interaction testing

Section B Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	10

Total Section B = 10 Marks

SECTION C — Scenario-Based Practical (50 Marks)

Purpose of Section C

Assess full PM-04 competency:

Test or debug source code to ensure client needs are met

Learners must design, debug, validate, test, and document results.

SECTION C.1 — User Interface Design (10 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Correct UI controls selected	2	IAC0101	Functional test readiness
Correct control naming	2	IAC0101	Interface clarity
Layout alignment with specification	3	IAC0101	UI correctness verification
Proper component usage	3	IAC0101	Interface usability validation

Mapping Summary

Criterion	Marks
IAC0101 Functional & non-functional tests identified	10

SECTION C.2 — Button Event Logic Implementation (20 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Add button functionality	8	IAC0102	Record insertion testing
Delete button functionality	7	IAC0102	Conditional logic validation
Find button functionality	5	IAC0102	Search behaviour verification

Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	20

SECTION C.3 — Software Testing Report (15 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Application description	3	AK0102	Documentation understanding
Test cases table	5	IAC0102	Test execution structure
Identified defects	4	IAC0101	Fault detection ability

Recommendations	3	IAC0103	Improvement reporting
-----------------	---	---------	-----------------------

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Branch created and used	1	IAC0102	Version control workflow
Professional commit messages	1	IAC0102	Change tracking clarity
Multiple commits recorded	1	IAC0102	Development progression evidence
Branch merged into main	1	IAC0102	Integration workflow
Repository complete & accessible	1	IAC0103	Submission verification

SECTION C Mapping Summary

Criterion	Marks
IAC0101 Tests identified	14
IAC0102 Tests applied	26
IAC0103 Test report produced	10

Total Section C = 50 Marks

FINAL OVERALL PM-04 ASSESSMENT SUMMARY

QCTO Criterion	Description	Marks
AK0101	Knowledge of testing techniques	25
AK0102	Knowledge of test reporting	18
IAC0101	Functional & non-functional tests identified	14
IAC0102	Correct tests applied	36
IAC0103	Test report produced	7

TOTAL = 100 MARKS